

DAUBES Ethan

DE BAYNAST Eloi

MARVIN Leo

VALAT Lou

Projet Gadz'Air
Conception d'un drone

Date de rendu : 07/12/2025

Table des matières

1.	INTRODUCTION	4
2.	MISE EN SITUATION	5
2.1.	OBJECTIF GENERAL DU PROJET	5
2.2.	PRESENTATION DES PARTIES PRENANTES	5
2.3.	CAHIER DES CHARGES FONCTIONNEL FOURNI PAR LE COMMANDITAIRE	5
2.4.	TRAVAUX DEJA REALISES SUR LE SUJET	5
3.	ORGANISATION DU PROJET	6
3.1.	DEFINITION DU PLANNING PREVISIONNEL	6
4.	DEFINITION DU PERIMETRE DU DRONE	7
4.1.	CAHIER DES CHARGES FONCTIONNELLES CONSOLIDE.....	7
4.1.1.	Redéfinition éventuelle de la fonction de service	10
4.1.2.	Recherche de contraintes supplémentaires.....	10
5.	CONCEPTION DU DRONE.....	11
5.1.	DETERMINATION DES SOLUTIONS ELEMENTAIRES	11
5.1.1.	Détermination des Fonctions techniques	11
5.1.2.	Choix des solutions	11
5.2.	DEFINITION DE L'ARCHITECTURE GENERALE.....	12
5.3.	DEFINITION DE LA STRUCTURE.....	12
5.4.	CHOIX DES COMPOSANTS.....	13
5.4.1.	Carte de vol	13
5.4.2.	Raspberry Pi 4.....	14
5.4.3.	Le module GPS PA1010D/ADA4415	15
5.4.4.	Bras du drone	15
5.5.	PRECONCEPTION DU SYSTEME D'INTELLIGENCE ARTIFICIELLE.....	16
5.5.1.	Définition de l'objectif de l'IA	17
5.5.2.	Recherche des données	17
5.5.3.	Présentation du code informatique utilisé	18
5.5.4.	Présentation du système de simulation.....	18
5.5.5.	Conclusion	19
5.6.	REPRESENTATION 3D DU SYSTEME	19
6.	FINANCEMENT.....	21
7.	REALISATION	22
7.1.	APPROVISIONNEMENTS / ACHATS DE COMPOSANTS	22
7.2.	FABRICATION DE COMPOSANTS	22
8.	CONCLUSION	23
	ANNEXE.....	24

ANNEXE 1 : CAHIER DES CHARGES FOURNI PAR DASSAULT	24
ANNEXE 2 : CODE PYTHON POUR DETECTION ET RECONNAISSANCE	25
ANNEXE 3 : LISTE DES COMPOSANTS.....	27
ANNEXE 4 : CODE INFORMATIQUE PERMETTANT DE DECIDER SI L'OBSTACLE EST DANGEREUX	28

1. Introduction

Dans le cadre de la compétition Dassault UAV Challenge, l'équipe Gadz'Air travaille à la conception d'un drone. Cette compétition, qui en est à sa 12^e édition, rassemble des élèves de différentes écoles et les invite à concevoir un drone intégrant plusieurs fonctionnalités autonomes.

L'objectif de ce rapport est donc de présenter l'état d'avancement du drone développé par l'équipe Gadz'Air. En effet, ce projet s'étend sur une année complète, avec une première phase de sélection prévue en décembre, suivie de la compétition finale en mai 2026.

En réponse à un cahier des charges fixés par Dassault, l'équipe Gadz'Air, composée d'Ethan Daubes, d'Eloi de Baynast, de Leo Marvin et de Lou Valat, a rédigé un rapport qui aura pour objectifs d'adresser les besoins suivants :

- Etablissement du cahier des charges
- Proposer une architecture de drone
- Dimensionner les composants requis
- Distribution du travail
- Budget prévisionnel

2. Mise en situation

2.1. Objectif général du projet

L'objectif est de créer un drone en vue de participer à la 1^{ère} édition de la compétition DASSAULT UAV Challenge. Sa fonction sera un drone avec une intelligence artificielle intégrée permettant de réaliser des missions, comme de la reconnaissance d'image et de la prise de décisions afin d'éviter des obstacles.

2.2. Présentation des parties prenantes

Les parties prenantes sont toutes les personnes et les services d'entreprises qui participent au projet. Il s'agit notamment de :

- La maîtrise d'ouvrage : l'entreprise Dassault, Dassault Aviation est une entreprise aéronautique française qui bénéficie d'une double expertise reconnue dans la fabrication d'avions militaires et d'avions d'affaires.
- La maîtrise d'œuvre : l'équipe Gadz'Air, équipe de 4 élèves présentée en introduction, élèves de l'Ecole Nationale Supérieure des Arts et Métiers du campus de Châlons en Champagne.

2.3. Cahier des charges fonctionnel fourni par le commanditaire

Le cahier des charges fourni par le commanditaire est fourni en Annexe 1, cependant on peut retenir les éléments principaux de celui-ci :

1. UAV – Spécifications de base :
 - a. Conformité avec la réglementation DGAC.
 - b. Système de suivi (tracking system) opérationnel.
 - c. Présence d'un interrupteur d'arrêt d'urgence (« kill switch ») manuel.
 - d. Déclencheur manuel d'annulation de mission opérationnel.
 - e. La perte de signal ne doit en aucun cas entraîner la perte du UAV.
2. Trajectoire :
 - a. Le UAV doit rester en permanence dans la zone de vol définie dans les annexes.
 - b. Le UAV doit être capable de suivre la trajectoire A (annexes) avec une enveloppe maximale de 3 mètres.
3. Autonomie & Intelligence :
 - a. Le UAV doit être capable de suivre un plan de vol de façon entièrement autonome.
4. Aspects liés au projet :
 - a. Le coût total du projet doit être inférieur à 1000 €.
 - b. Une facture détaillée de l'ensemble des composants utilisés (précision au centime) doit être fournie.
 - c. Un résumé complet du challenge doit être inclus dans le dossier.

2.4. Travaux déjà réalisés sur le sujet

Aucun travail n'a encore été réalisé sur le sujet, l'équipe Gadz'Air part entièrement de 0.

3. Organisation du projet

3.1. Définition du planning prévisionnel

Le planning ci-dessous se termine janvier, date à laquelle la deuxième équipe du campus des Arts et Métiers prendra le relai sur le projet, cependant les membres inscrit au concours, continueront à superviser le projet afin de l'apporter jusqu'à la compétition. Les numéros sur le planning correspondent aux dates de rendus de nos rapports auprès de nos professeurs afin de voir notre avancement sur le projet, nous avons mis que les 3 derniers rendus. Enfin la lettre D sur le planning correspond au rendu du rapport que vous êtes actuellement en train de lire.

PLANNING PRÉVISIONNEL PROJET DASSAULT													
		2025								2026			
MACRO-TACHES	TACHES	45	46	47	48	49	50	51	52	1	2	3	
Propulsion	Choix moteurs												
Batterie	Bilan de conso. Électrique composants												
	Choix batterie de vol / composants												
	Achat matériel de recharge												
Structure	Référencement des pièces à imprimer												
	Conception du modèle 3D adadpté aux attendus												
	Analyse RDM sur structure modélisée												
	Commande du PLA renforcé ou Alu.												
Télémétrie	Choix télécommande de contrôle de vol												
	Choix Antenne de connexion / Antenne vidéo												
IA	Implémentation et mise en place du retour vidéo												
Vol autonome	Configuration carte IA												
	Configuration carte de contrôle de vol												
	Prise en main logiciel vol autonome												
	Configuration et installation du hardware												
Assemblage	Achat matériel												
	Montage des composants sur structure												
	Ajout des composants électroniques												
Réglementation	Formation DGAC												
Essais	Validation fonct. général et respect CDCF												

4. Définition du périmètre du drone

Pour définir les besoins précis de notre système ainsi que les composants et programmes nécessaires au bon fonctionnement, il nous faut connaître les conditions de vol de notre drone ainsi que son environnement si possible.

En effet, la maîtrise d'ouvrage nous précise que le concours aura lieu en extérieur sur un grand espace et que le drone ne sera pas soumis à un grand nombre d'obstacles naturels.

Toutefois, il faudra prendre en compte le fait que le jour du concours, les conditions météo peuvent être mauvaises (rafales de vents, pluie, orage ...). Il ne faut donc pas négliger ces contraintes faisant partie de l'environnement du système.

4.1. Cahier des charges fonctionnelles consolidé

Dans le but de décrire au mieux les objectifs ainsi que les attentes et les critères souhaités, la consolidation d'un cahier des charges est nécessaire. Il permet ainsi une compréhension plus facile du projet par les parties prenantes.

Issu des attentes du concours ainsi que des caractéristiques souhaitées par notre équipe, il synthétise l'essentiel des informations nécessaires pour cadrer le projet. Ainsi, le cahier des charges consolidé est présenté ci-dessous :

Secteur	Critère / Fonction de service	Description	Exigence (Valeur ou critère)	Exigence détaillée	Priorité	Solutions technologiques	Méthode de vérification	Remarques
Structure / Mécanique	Masse totale	Masse au décollage (MTOW) incluant batterie et charge utile		≤ 2.5 kg (à ajuster selon la catégorie de la compétition)	Haute		Pesée sur balance	Respecter les limites réglementaires et de compétition
Structure / Mécanique	Rigidité châssis	Résistance aux vibrations et chocs lors d'atterrissage		Pas de fissure après tests d'impacts légers	Moyenne	Solution : châssis en alliage d'aluminium ET OU avec fibre de carbone	Essai mécanique + inspection visuelle	Discuter avec technicien pour aide au choix
Structure / Mécanique	Fixations	Système d'accrochage hélices/moteurs et charges utiles		Sécurité : pas de desserrage en vol	Haute		Inspection + tests en vol	
Propulsion	Moteurs	Type et nombre de moteurs		Brushless, 4 moteurs (pour un quadricoptère)	Haute		Vérification fiche technique + essai au banc	
Propulsion	Hélices	Taille / pas des hélices		Concordance moteur/hélice recommandée par datasheet	Moyenne		Mesure poussée statique	
Propulsion	Poussée totale	Poussée disponible pour monter et manœuvres		Poussée maximale > 2× masse en N (safety margin)	Haute		Test poussée statique	
Énergie / Batterie	Type batterie	Chimie, capacité et connectique	Capacité cible 5000-12000 mAh	LiPo 4S/6S selon motorisation,	Haute	Batterie Lithium NCM OU Nickel / Cadmium	Vérification spec + test d'endurance	
Énergie / Batterie	Autonomie	Durée de vol en conditions de mission	Target : 20 min	≥ 20 minutes en profil mission réel (avec marge)	Haute		Test vol réel + log télémétrie	
Énergie / Batterie	Sécurité batterie	Protection contre surcharge, court-circuit, incendie		BMS ou circuit de protection + procédures de stockage	Haute		Inspection, test de charge/décharge	
Avionique / Capteurs	Contrôleur de vol	Capacité d'autopilot, GPS, IMU		Contrôleur avec firmware open- source	Haute	Ardupilot	Vérification configuration + logs	Récupérer les données à distances
Avionique / Capteurs	GPS	Positionnement pour navigation autonome	Précision à 2m	GPS RTK si besoin ; précision < 2 m sans RTK	Moyenne		Test précision GPS au sol/vol	https://docs.centipede.fr/docs/centipede/2_RTK.html
Avionique / Capteurs	Capteurs supplémentaires	Baromètre, compas, optique (flow/vision) si nécessaire		Baromètre pour maintien d'altitude, optical flow pour intérieur	Moyenne		Tests fonctionnels en vol	
Télécommuni- cations	Télémétrie	Lien télémétrie bidirectionnel		Portée utile ≥ 1 km en champ libre	Haute		Test de portée en champ libre	Exigence à revoir si besoin
Télécommuni- cations	Lien de commande	Robustesse du lien RC		Fréquence et redondance selon règles	Haute		Test de latence et liaison	
Logiciel / Contrôle	Modes de vol	Manuel, stabilisé, autonome (waypoints)		Doit supporter plan de vol autonome et override manuel	Haute		Test en simulation + vol réel	
Logiciel / Contrôle	Algorithmes	Guidage, navigation, évitement d'obstacles		Évitement passif/actif	Moyenne		Simulation + tests terrain	
Logiciel / Contrôle	Logs & Télémétrie	Enregistrement des paramètres de vol		Logs complets (IMU, GPS, commandes) pendant chaque vol	Haute		Vérification des fichiers de logs	Enregistrement sur ordinateur à distance ?
Charge utile	Caméra / Capteurs mission	Type de capteur embarqué (caméra RGB, multispectral, etc.)		Poids et alimentation compatibles	Moyenne		Test intégration + qualité image	

Charge utile	Capacité transport	Capacité à embarquer un objet pour la compétition	300 g	≥ 300 g utile (exemple)	Moyenne		Test montée/vol avec charge	
Sécurité / Réglementation	Parachute / système de secours	Optionnel		Si présent : déploiement < 3 s	Basse		Test au sol + procédures	
Sécurité / Réglementation	Conformité réglementaire	Règles de l'air locales et règles compétition		Respect des limites de masse, zones & fréquences	Haute		Vérification documentaire	
Sécurité / Réglementation	Interrupteur d'arrêt	Coupe-moteurs accessible	1 s de réaction	Kill switch accessible télécommande + bouton au drone	Haute		Test fonctionnel	
Maintenance	Accessibilité	Facilité de démontage et maintenance		Remplacement d'un bras en < 15 min	Moyenne		Test démonstration	
Maintenance	Documentation	Manuels, procédures et checklists		Checklist pré-vol et maintenance disponible	Haute		Existence des docs	
Interface / Opération	Outils sol	Station sol pour plan de vol et télémétrie		Interface graphique pour planification de missions	Moyenne		Test utilisation pendant vol	
Interface / Opération	Ergonomie télécommande	Lisibilité et accès aux commandes critiques		Affichage clair télémétrie + boutons dédiés	Moyenne		Retour opérateur	
Planning & Budget	Délais projet	Jalons : conception, prototype, test, livraison	7 décembre	Rapport de conception décembre ; compétition potentielle en mai	Haute		Planification Gantt	
Planning & Budget	Coût cible	Budget matériel total	≤ 1000 €		Moyenne		Devis fournisseurs	
Approvisionnement	Fournisseurs critiques	Moteurs, ESC, contrôleur de vol, batteries	Voir détail des fournisseurs		Moyenne		Liste fournisseurs + lead times	
Fiabilité	MTBF estimé	Durée moyenne avant défaillance		MTBF > 100 heures pour composants critiques	Moyenne		Calculs + retours fournisseurs	
Performances	Vitesse maximale	Vitesse de déplacement	$V_{\max} = 19 \text{ m.s}^{-1}$	≥ 19 m/s en pointe (selon mission) (autour de 70kmh ⁻¹)	Moyenne		Essai en vol	
Performances	Taux de montée	Capacité d'ascension		≥ 3 m/s	Moyenne		Essai en vol	
Structure / Mécanique	Masse totale	Masse au décollage (MTOW) incluant batterie et charge utile		≤ 2.5 kg (à ajuster selon la catégorie de la compétition)	Haute		Pesée sur balance étalonnée	Respecter les limites réglementaires et de compétition
Structure / Mécanique	Rigidité châssis	Résistance aux vibrations et chocs lors d'atterrissage		Pas de fissure après tests d'impacts légers	Moyenne	Solution : châssis en alliage d'aluminium ET OU avec fibre de carbone	Essai mécanique + inspection visuelle	Discuter avec technicien pour aide au choix
Tests & Validation	Bancs d'essais	Essais moteurs, ESC, alimentation		Banc moteur + enregistrements	Haute		Rapports d'essais	
Tests & Validation	Campagne de vols	Phases d'essais progressives		Séances en intérieur puis extérieur, montée en complexité	Haute		Journal de vols + logs	

4.1.1. Redéfinition éventuelle de la fonction de service

Le drone Gadz'Air ne remplira pas uniquement les prérequis fournis par le commanditaire, sa fonction de service sera le vol autonome avec une intelligence artificielle. Son but est de, par soi-même, analyser les obstacles que la caméra perçoit, les identifier et les analyser, puis à l'aide d'une matrice de décisions et des programmes

4.1.2. Recherche de contraintes supplémentaires

Dans le cadre de la conception du drone, nous avons dû renforcer le cahier des charges qui a été fourni comme fil directeur. Nous avons ajouté des valeurs numériques suivant les critères imposés par Dassault mentionnés précédemment dans le cahier des charges afin de pouvoir quantifier les objectifs que nous nous sommes fixés et voir si nous les avons atteints.

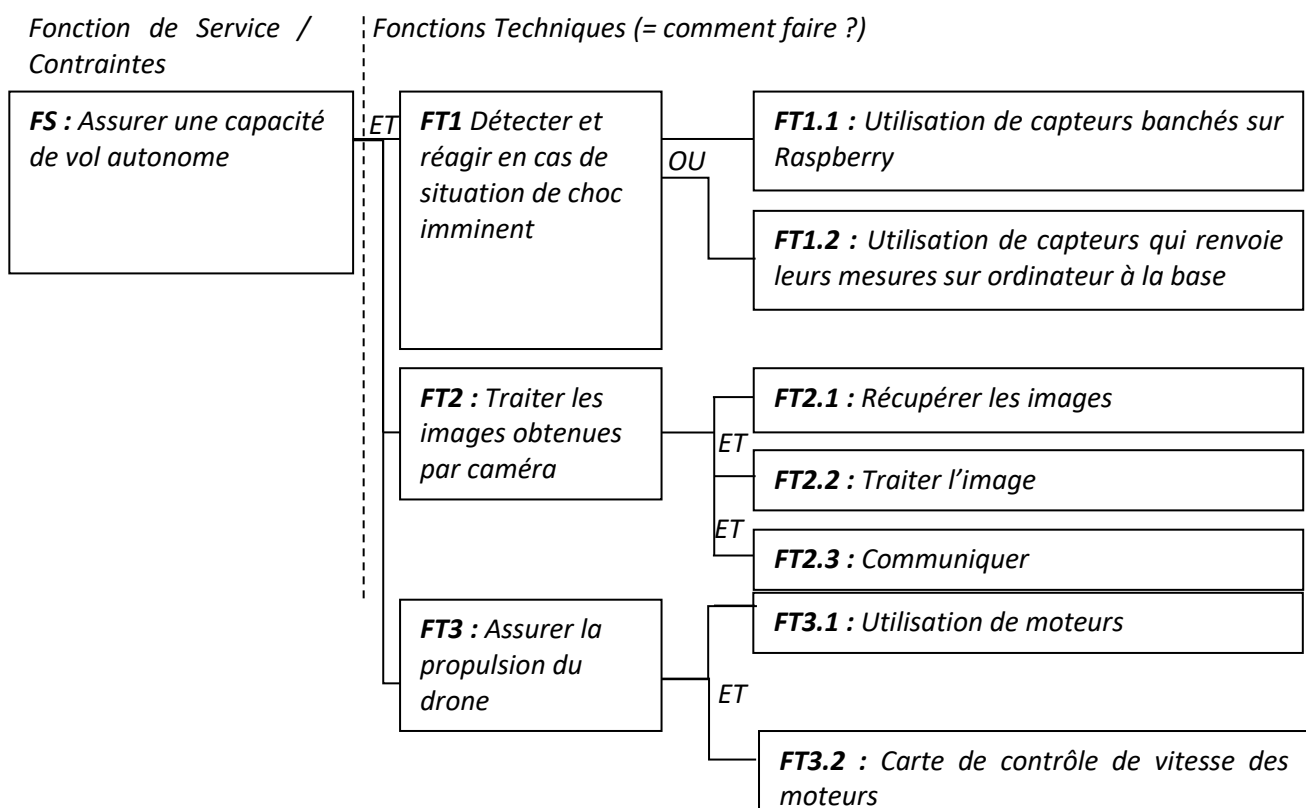
5. Conception du drone

Dans le cadre de la compétition, il est essentiel de pouvoir connaître les caractéristiques du drone. En effet, pour pouvoir répondre au cahier des charges, nous devons, avant de les assembler pouvoir fabriquer les différents composants, que ce soient des pièces qui seront physiquement montées sur le drone ou des codes informatiques à développer pour assurer que la mission du drone est remplie.

5.1. Détermination des solutions élémentaires

5.1.1. Détermination des Fonctions techniques

Afin de pouvoir résumer les différentes fonctions techniques réalisées par le drone, nous allons nous baser sur un FAST (Function Analysis System Technique)



5.1.2. Choix des solutions

5.1.2.1. Définition et hiérarchisation des critères de choix généraux

Dans le cadre de notre cahier des charges, nous pouvons en tirer plusieurs critères de choix qui seront décisifs dans le cas des choix que nous devons faire dans le cas des solutions technologiques. Ces critères sont les suivants :

- Durée de vol
- Résistances aux chocs
- Coût cible total inférieur à 1000 €
- Fonctionnalité (capable d'atteindre les performances fixées par le cahier des charges)

- Masse

Cependant, tous ces critères n'ont pas la même importance. Notre projet impose que la fonctionnalité des composants est une priorité. De plus, étant limité par la capacité des moteurs, nous devons de chercher à constamment optimiser la masse

5.2. Définition de l'architecture générale

Les choix ont été multiples pour le drone, on a tout d'abord pensé à un drone à aile plane de surveillance, des recherches et des simulations rapides sur la portance ont été effectuées. L'objectif initial était de réaliser un drone « furtif » à aile plane qui pourrait survoler de nuit des zones, et à l'aide d'une caméra thermique, faire une reconnaissance d'un lieu et de réaliser des missions de surveillance. Cependant dû à de nombreux éléments assez complexes comme le décollage et l'atterrissage automatique, le prix de la caméra infrarouge haute définition, ou encore la dangerosité de faire voler un avion qu'on ne voit pas de nuit, le projet fut écarté. Le choix de notre équipe c'est donc porté sur un quadricoptère couplé à de l'intelligence artificielle, car nous avons considéré que c'était plus simple et correspondait plus à l'avenir des drones. Nous avons suivi le modèle X500 comme base de drone, car celui-ci est suffisamment grand pour pouvoir accueillir en son sein une carte NVIDIA Orion Jetson Nano, qui a été choisie pour héberger notre IA du fait de son ratio prix et puissance de calcul.

Le choix de notre équipe se porte donc sur un drone quadricoptère avec 2 pieds de maintien afin de garantir un atterrissage en douceur, et de tenir à l'écart du sol les différents composants électroniques. Le châssis sera composé de 2 plaques en parallèles, où nous pourrions commencer à mettre une partie des composants entre les deux plaques et la carte NVIDIA sur la plaque supérieure.

5.3. Définition de la structure

La structure utilisée sera celle du drone X500, car celui-ci est assez grand pour accueillir la carte d'IA, et de plus il y a assez de place pour mettre une grande quantité de composant et de manière assez modulable. La structure est composée de quatre bras cylindriques creux fixés par pincement à leur extrémité. Au bout de ces bras on a de l'espace pour accueillir les moteurs voulus, et avec le câblage qui pourra passer à l'intérieur des bras. L'îlot central est composé de 2 plaques, une inférieure qui permettra d'accueillir les composants comme le GPS, l'ordinateur de vol ou autre. La plaque supérieure accueillera la carte d'IA. Les batteries seront fixées en dessous de la plaque inférieure sur le support prévu à cet effet. En addition la caméra du drone sera fixée sur le support sur le côté du drone. Enfin, 2 pieds permettront d'assurer la fonction du train d'atterrissage du drone.



Figure 1 Image de la structure retenue

5.4. Choix des composants

5.4.1. Carte de vol

Le choix du contrôleur de vol s'est porté sur la RadioLink Pix6, en raison de son équilibre entre fiabilité, performance et compatibilité logicielle. La carte constitue une solution robuste adaptée à un drone autonome tout en maîtrisant les coûts.

La RadioLink PIX6 est nativement supportée par ArduPilot, logiciel de référence pour le vol autonome. Cela garantit :

- Une intégration immédiate du firmware,
- Des mises à jour fiables,
- La disponibilité de nombreux modes de vol (Stabilize, Auto, Loiter, RTL, failsafes),
- Un support communautaire et documentaire important qui permet une formation approfondie à la manipulation du logiciel.

Architecture matérielle adaptée

La carte intègre des capteurs inertiels redondants, un baromètre précis et une alimentation stabilisée. Ses connecteurs JST-GH standardisés permettent un câblage propre et rapide (GPS, télémétrie, récepteur radio, ordinateur embarqué). Ceci réduit les risques d'erreur et assure une fiabilité indispensable pour les missions autonomes.

Évolutivité pour les développements futurs

Grâce à ses ports UART et USB, la RadioLink PIX6 permet l'intégration d'un companion computer (ex. Raspberry Pi) pour la vision embarquée ou des traitements avancés, répondant ainsi aux besoins d'un projet intégrant potentiellement de l'IA.

Rapport coût/fiabilité

Positionnée entre les contrôleurs hobby et les solutions professionnelles plus coûteuses, la RadioLink PIX6 offre un très bon compromis pour un projet nécessitant précision et sécurité sans dépasser le budget.

Mission Planner est l'outil principal de configuration, calibration et planification utilisé avec ArduPilot.

Son choix est justifié par plusieurs points : Mission Planner reconnaît automatiquement la carte, facilite l'installation du firmware et la configuration des paramètres sans nécessiter de manipulation avancée.

Le logiciel permet :

- La création de trajectoires complexes par waypoints (points clés),
- La définition de vitesses, altitudes et actions spécifiques,
- La simulation du vol avant exécution.

Cela répond directement aux exigences de vol autonome du cahier des charges.

Mission Planner offre :

- Affichage des capteurs, batteries, moteurs, GPS,
- Enregistrement des logs de vol,
- Outils de vérification pré-vol (failsafe, redondance IMU, qualité GPS),
- Analyse post-vol.

Ces éléments sont indispensables pour la validation, la mise au point et la fiabilité du système.

Mission Planner communique en MAVLink, standard également utilisé par la RadioLink PIX6 et les calculateurs embarqués (Raspberry Pi). Ceci permet une intégration cohérente de la télémétrie, de la supervision longue portée ou de modules IA.

En conclusion, la **RadioLink PIX6**, associée au logiciel **ArduPilot** et à l'outil **Mission Planner**, constitue une solution cohérente, économique et techniquement adaptée à un drone autonome. Elle offre :

- Une fiabilité matérielle suffisante,
- Un environnement logiciel complet,
- Une excellente évolutivité,
- Une facilité d'intégration compatible avec les objectifs du projet.

5.4.2. Raspberry Pi 4

La Raspberry Pi 4 a été choisie comme ordinateur embarqué car elle offre une puissance de calcul largement suffisante pour traiter en temps réel le flux vidéo de la caméra, assurer l'envoi d'images, gérer d'éventuels traitements embarqués et communiquer efficacement avec le contrôleur de vol Pixhawk. Ses interfaces intégrées (USB, UART, CSI pour caméra, Wi-Fi et Ethernet) permettent une intégration simple et flexible, tout en limitant le nombre de modules supplémentaires. Cette capacité à traiter et transmettre des données en continu répond directement aux besoins du drone : retour vidéo en temps réel, suivi de mission, et transmission de télémétrie.

Son format compact, sa consommation raisonnable et la disponibilité d'un large écosystème logiciel en font une solution fiable et évolutive, particulièrement adaptée à un projet nécessitant autonomie, supervision à distance et possibilité de développement personnalisé. La Raspberry Pi 4

permet également de séparer clairement les tâches : le Pixhawk gère le pilotage et la sécurité (stabilisation, RTL, modes de vol), tandis que la Pi prend en charge la partie “intelligente” du drone. Ce découpage améliore la fiabilité globale du système et s’aligne parfaitement avec le cahier des charges du drone autonome.

5.4.3. Le module GPS PA1010D/ADA4415

Le module GPS PA1010D/ADA4415 a été choisi pour équiper le drone car il offre une bonne précision de positionnement et une acquisition rapide des satellites grâce à son antenne céramique intégrée. Sa compatibilité multi-GNSS (GPS, Galileo, GLONASS) garantit une localisation fiable, essentielle pour le suivi autonome du plan de vol, le maintien de position et la fonction Return To Launch en cas de perte du lien de commande. Son format compact, sa faible consommation et son interface UART facilitent son intégration avec le contrôleur de vol Pixhawk, déjà retenu dans l’architecture électronique du drone.

L’absence de boussole intégrée n’est pas un inconvénient, car elle permet au contraire d’utiliser un compas externe séparé, moins sensible aux perturbations électromagnétiques générées par les moteurs et l’électronique de puissance. Cette séparation GPS/compas constitue une bonne pratique dans les systèmes ArduPilot, améliorant la fiabilité du cap et la stabilité en vol. Le PA1010D/ADA4415 répond ainsi aux critères de fiabilité, précision et compatibilité imposés par le cahier des charges du drone autonome.

5.4.4. Bras du drone

Comme exposé précédemment dans ce rapport, le drone s’appuie sur une structure à 4 bras (embarquant chacun un moteur) afin de mettre en place la propulsion globale de celui-ci. Chaque bras va ainsi subir différentes contraintes issues des efforts / couples générés par le moteur situé à son extrémité. On obtient la représentation ci-dessous :

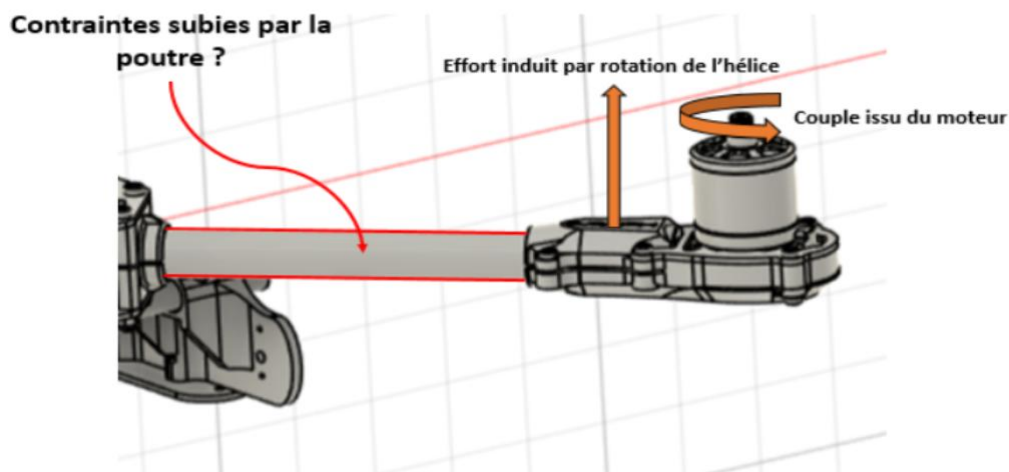


Figure 2: Modélisation de la situation étudiée

L’objectif principal étant la construction complète du modèle de drone considéré, il faut se pencher sur les matériaux employés pour construire les différents composants de structure nécessaires. Dans le cas du bras, une étude de Résistance Des Matériaux (RDM) est nécessaire pour

vérifier, selon les caractéristiques du drone (dimensions / cahier des charges ...), la validité d'emploi d'un matériau pour réaliser chaque bras du drone.

L'étude s'articulera de la manière suivante :

- Mise en place du modèle pour simulation et initialisation de la phase de calculs
- Mise en place des hypothèses pour simplifier le modèle déterminé
- Application des méthodes de résolution classiques ou simulées pour obtention de résultats
- Conclusion sur choix du matériau vis-à-vis des résultats obtenus

Dans l'avancement actuel du projet, la mise en place du modèle a été effectuée et la connaissance des données pertinentes pour les calculs à venir nous permet actuellement de nous pencher sur la prise d'hypothèses afin d'ensuite réaliser les calculs de prédimensionnement. Pour l'instant les réflexions se portent sur les éléments suivants :

A - Bras assimilé à une poutre (OK)

B - Recensement des efforts (Choix du type d'efforts pour une application simple -> EN COURS)

C – Choix de la méthode de calcul pour son application par la suite (À SUIVRE).

Sollicitation	N	T	$\mathcal{M}_t$	$\mathcal{M}_f$	Contraintes
Traction simple	$\neq 0$	$= 0$	$= 0$	$= 0$	$\sigma_{11} \neq 0$ et $\vec{\tau} = 0$
Cisaillement simple	$= 0$	$\neq 0$	$= 0$	$= 0$	$\sigma_{11} = 0$ et $\vec{\tau} \neq 0$
Torsion simple	$= 0$	$= 0$	$\neq 0$	$= 0$	$\sigma_{11} = 0$ et $\vec{\tau} \neq 0$
Flexion pure	$= 0$	$= 0$	$= 0$	$\neq 0$	$\sigma_{11} \neq 0$ et $\vec{\tau} = 0$
Flexion simple	$= 0$	$\neq 0$	$= 0$	$\neq 0$	$\sigma_{11} \neq 0$ et $\vec{\tau} \neq 0$
Flexion composée	$\neq 0$	$\neq 0$	$= 0$	$\neq 0$	$\sigma_{11} \neq 0$ et $\vec{\tau} \neq 0$
Sollicitation composée	$\neq 0$	$\neq 0$	$\neq 0$	$\neq 0$	$\sigma_{11} \neq 0$ et $\vec{\tau} \neq 0$

Figure 4 : 3 Liste des sollicitations possibles appliquées à la poutre

Une fois ces éléments fixés, nous serons en mesure de réaliser les calculs et de choisir le matériau adapté pour les bras de la maquette finale du drone. Le drone sera alors pleinement en mesure d'effectuer le plan de vol souhaité avec des bras résistant aux contraintes générées par le moteur en toute circonstance.

5.5. Préconception du système d'intelligence artificielle

Dans un contexte où l'intelligence artificielle occupe une place croissante, nous avons choisi de nous inscrire dans cette évolution en concevant un drone dont la caractéristique principale est l'intégration de l'IA au système de vol, lui permettant notamment de pouvoir voler de manière autonome une fois qu'une mission est donnée. De plus, nous incorporons sur notre drone un système

qui traite les images captées par la caméra du drone avant que celles-ci ne soient renvoyées vers la Home base au moyen d'un ordinateur.

5.5.1. Définition de l'objectif de l'IA

Dans l'objectif précisé plus haut, nous allons dans un premier temps, chercher à pouvoir concevoir le système qui détecte et classe les différents objets vus par la caméra. Nous avons fait le choix de commencer par cette partie puisque nous avons besoin de pouvoir catégoriser les obstacles avant que le drone ne doive décider si oui ou non l'obstacle peut être considéré comme dangereux. Dans un second temps, nous utilisons un système qui se base sur la différence entre deux images successives pour pouvoir décider ou non si l'obstacle est dangereux et pouvoir suggérer une correction possible pour l'éviter.

5.5.2. Recherche des données

D'abord, nous avons besoin avant de pouvoir commencer à concevoir notre système numérique de considérer les différentes possibilités. En effet, nos recherches nous ont apportés à trouver plusieurs possibilités résumées dans le tableau suivant :

Solution	Type	Avantages	Limites	Application	Mise en application
OpenCV	Vision classique (détection, suivi)	- Léger, - Possède des algorithmes déjà écrits utilisables	Pas de Deep Learning	Détection et suivi de cibles	Facile à mettre en œuvre sur Raspberry Pi ou Jetson Nano
YOLO (ultralytics)	Vision classique (détection, suivi)	Bon compromis précision – vitesse	Ne fonctionne pas sur Raspberry aussi bien que sur Jetson	Détection et suivi de cible	Optimisé pour Jetson
PyTorch	Deep Learning	Capable d'être couplé à un autre logiciel grâce à son application programming interface (API)	Besoin de données pour entraîner le modèle	Sert à entraîner un réseau de neurones pour détecter des objets, les classer, naviguer en se basant sur ce que le drone doit accomplir	Développer sur ordinateur le modèle qu'on exporte ensuite sur Jetson
TensorFlow	Deep Learning avec entraînement et mise en place sur système non puissant	Ecosystème complet	- Système lourd - Complicé à aborder	- Détection/segmentation, - Classification, - Optimisation des modèles embarqués	Requiert système d'adaptation à la carte Jetson ou Raspberry

5.5.3. Présentation du code informatique utilisé

Nous avons joint en annexe le code Python qui permet de récupérer la vidéo et proposer un traitement de celle-ci pour en reconnaître son contenu ce qui sera ensuite utilisé afin de pouvoir décider si l'obstacle est dangereux.

Ce code se base sur l'utilisation de YOLOv8n, solution issue du modèle YOLO disponible sur Python. Ce modèle permet de détecter un objet et le classer parmi une liste d'option. Pour détecter les objets, nous faisons appel à la fonction model que l'on applique à une image. Cette fonction renvoie les coordonnées du rectangle des différents objets ainsi que l'identifiant de cet objet pour le classer et un taux de confiance dans l'identification.



Figure 4 : Exemple d'application du traitement informatique pour caractériser les éléments d'une image

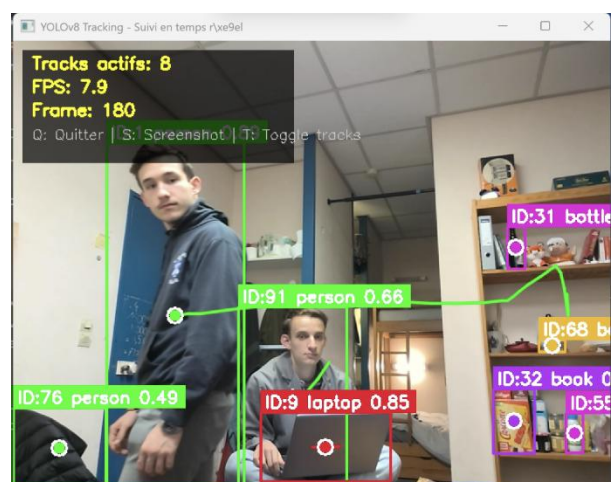


Figure 5 : Application du traitement informatique pour suivre une cible

En parallèle, nous avons développé, grâce à YOLO, un système permettant de déterminer si un obstacle doit être considéré comme dangereux. Ce code, présenté en annexe, repose sur la comparaison de deux images successives. Il produit une image dans laquelle les zones dont le déplacement est trop important sont mises en surbrillance et considérées comme dangereuses, car elles se rapprochent trop rapidement.

Ce module, que nous venons d'achever, n'a pas encore pu être intégré au drone, mais il a été testé dans un premier temps grâce à la transmission en temps réel du flux vidéo d'un téléphone et offre des perspectives prometteuses dans le cas de notre drone.

5.5.4. Présentation du système de simulation

Avant de pouvoir passer à la construction du drone et la mise en service de l'intelligence artificielle, nous avons fait le choix de tester de manière numérique notre système dans l'optique de réduire les coûts en évitant ainsi de construire plusieurs drones.

Pour se faire, nous allons utiliser le logiciel AirSim qui génère une simulation sur Unreal Engine. AirSim est un logiciel développé par Microsoft qui permet de générer un environnement dans lequel nous pouvons mettre en œuvre notre intelligence artificielle.

Nous avons commencé à simuler un environnement virtuel, où nous avons simulé un drone avec notre IA à l'intérieur et on le met face à des situations qui pourraient arriver dans la réalité, et on voit comment il réagit en fonction de ces situations, cela lui permettra d'affiner et d'entraîner notre IA. Par exemple des évitements d'objets sur la trajectoire demandée.

5.5.5. Conclusion

Pour pouvoir résumer le fonctionnement du système de l'intelligence artificielle, nous proposons un arbre décisionnel qui présente les différentes étapes.

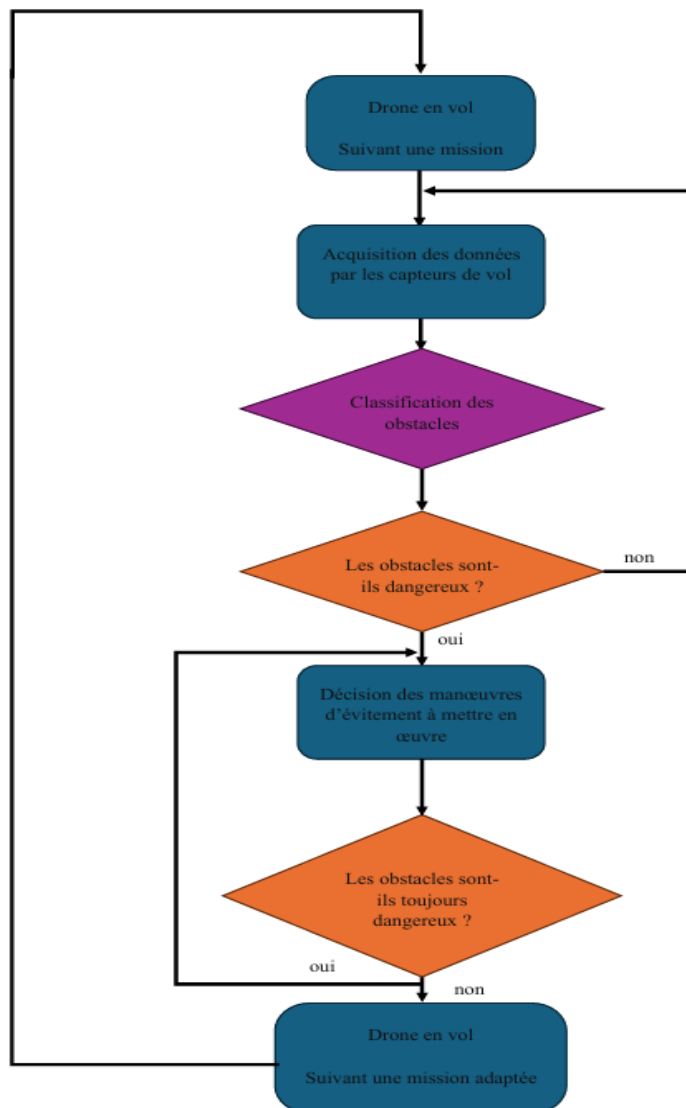


Figure 6 : Principe d'incorporation de l'IA dans le fonctionnement du drone

En somme, nous avons fait le choix lors de notre organisation du travail de privilégier la conception du système d'intelligence puisque nous avons considéré, après réflexion et des recherches, que cette étape était la plus compliquée et méritait donc d'être abordée en premier.

5.6. Représentation 3D du système

Dans l'idée de montrer que l'équipe Gadz'Air est capable de relever le défi posé par cette compétition, nous avons fait le choix de proposer une architecture du drone que nous cherchons à fabriquer. Pour ce faire, nous avons fait le choix de proposer une représentation en conception assistée par ordinateur (CAO) du système que nous allons fabriquer.



Figure 7 : Proposition d'architecture du drone

6. Financement

Les financements seront de l'ordre de 500€ de la part de l'administration de l'École Nationale Supérieure des Arts et Métiers du campus de Châlons-en-Champagne, ainsi, dans l'hypothèse de la sélection de l'équipe des Gadz'Air pour la compétition, un financement supplémentaire de 500€ de la part de l'entreprise Dassault Aviation. Nous proposons le budget prévisionnel ne prenant pas en compte la carte NVIDIA Orion Jetson Nano (en effet, nous avons fait le choix de tester les modules d'intelligence artificielle pour le moment avec une carte NVIDIA Orion Jetson qui nous a été prêtée par notre enseignant référent, non montable sur un drone car trop massive, et un Raspberry Pi pour éviter un investissement trop conséquent avant que nous choissions si oui ou non nous la conservons la Jetson Nano) :

Élément	PRIX (EN EUROS)
Caméra	21,8
Radiolink PIX6	185,45
Hélices	25,4
Chargeur de batterie	36,3
Raspberry pi 4	65,4
Batterie	44,33
Moteurs*4 (ici prix totales)	91,6
LIGPOWER	136,96
Module GPS PA1010D ADA4415 (pas de boussole)	40,32
télécommande émetteur + récepteur	59,99
	TOTAL
	707,55

7. Réalisation

Pour effectuer des tests ainsi que des simulations de vol et de traitement d'image, il est nécessaire de se procurer les éléments essentiels à l'assemblage du drone. Dans cet objectif, cette partie vise à détailler les achats effectués depuis le début d'année.

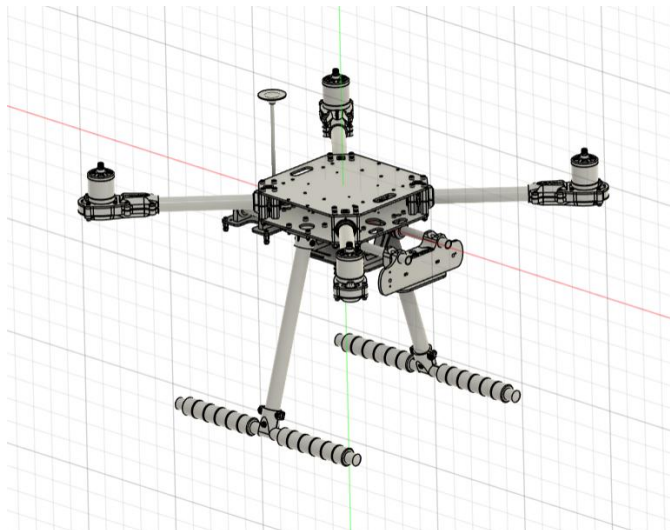
7.1. Approvisionnements / achats de composants

Afin de subvenir aux besoins de notre drone et pouvoir supporter le bon fonctionnement de l'IA et du traitement d'image, il est nécessaire de prévoir l'achat des composants étant donné que l'École Nationale Supérieure des Arts et Métiers de Châlons en Champagne ne permet pas d'achats de composants une fois le mois de décembre dépassé sauf quelques exceptions.

Possédant un partenariat avec l'école, l'entièreté des achats a été effectué sur le site Robotshop. La liste des composants achetés et obtenus sont mentionnés en annexe 3.

7.2. Fabrication de composants

Nous devons réaliser une maquette qui pourra maintenir les composants entre eux et donc créer la base du drone. Cette dernière a été inspirée d'une maquette déjà existante.



Il est maintenant question de savoir quelles pièces vont être fabriquées par quel moyen et dans quels matériaux. Tout d'abord, les plaques qui vont soutenir les composants numériques, va être réalisées par moulage avec sable puis avec usinage par une fraiseuse trois axes.

Pour ce qui est des arbres qui vont soutenir les moteurs, il est nécessaire de solidifier la structure avec du fil PLA renforcée au carbone ou encore, en vue des problèmes que rencontre le fil PLA envers les intempéries, il serait préférable de se tourner vers du composite à l'aide d'une machine comme la MacForge two.

Le reste des pièces pourrait quant à elles être simplement être obtenue par impression 3D avec du fil PLA, au moins pour la première maquette.

8. Conclusion

Pour conclure, l'IA est en cours de développement et est presque finie. Les différents modules de cet IA sont prêts à être intégré dans le drone et seront prochainement intégrés dans l'IA durant les prochaines semaines en décembre. L'équipe ayant réussi à prendre en main le logiciel et faire des simulations de vols, la partie vol autonome avec le logiciel ArduPilot est en cours de réalisation.

De plus, les composants ont été achetés et sont en cours de livraison. L'ensemble des différentes pièces structurelle du drone sont en cours d'impression.

Annexe :

Annexe 1 : Cahier des charges fourni par Dassault

Theme	Requirements	Level	Max points	Notes	Actual points
UAV's Basic Specifications	GD&C compliance	Mandatory	0	Test #1	
	Control system is operational	Mandatory	0	Test #1	
Safety	Manual emergency kill switch	Mandatory	0	Test #1	
	Manual mission cancellation trigger is operational	Mandatory	0	Test #1	
	In case of error, UAV automatically ends the mission	Important	1		
	Ending the mission leads to RTN	Nice to have	1		
	Ending the mission does not damage the drone	Nice to have	1		
	Loss of signal does not lead to loss of UAV	Mandatory	0	Test #1	
	Loss of signal triggers RTN automatically	Important	1		
	Real-time battery monitoring	Important	1		
	On-ground station	Nice to have	1		
	Mission can automatically be stopped once below a given battery capacity	Nice to have	1		
Trajectory	UAV must stay within the flight area defined in section IX of the appendix, at all time	Mandatory	0	This is mandatory at all times.	
	UAV can follow trajectory & from the appendix, within a 1 meter's margin	Mandatory	0	Test #1, 2) Trajectory accuracy must be proven by the team to the jury.	
	Within a 1 meter's margin	Important	1	3. Trajectory accuracy must be proven by the team to the jury.	
	Within a 10cm's margin	Nice to have	1	3. Trajectory accuracy must be proven by the team to the jury.	
	Within 1 minute	Important	1		
	Within 30 seconds	Nice to have	1		
	Quantify the uncertainty and performance of the positioning system	Important	1		
	UAV can fly for more than 10 minutes	Important	1		
Flight Time	UAV can fly for more than 30 minutes	Nice to have	1		
	UAV can fly virtually indefinitely	Nice to have	1		
Autonomous & Intelligence	UAV can follow a flight plan autonomously	Mandatory	0	Test #1	
	Along with a real-time thorough monitoring on ground station	Important	1	Battery level, speed, heading, mission status, real progress, 1 samples, etc.	
	Along with a real-time video from the UAV	Nice to have	1		
	Onboard computing	Nice to have	1	Ground station only receive data but does not do any computing.	
	UAV can guide itself by computer vision and/or pattern recognition	Nice to have	10		
	GPI board capacity using computer vision	Nice to have	10		
	Complex pattern recognition and/or classification and/or any machine learning technique	Nice to have	1		
	Reinforcement learning	Nice to have	10		
	Flown using UAV can take off autonomously	Nice to have	1		
	Within less than 10 meters	Nice to have	1		
	Virtually 3D/2D	Nice to have	1		
	Capacity to automatically generate a flight plan just before the drop and/or before take-off	Nice to have	10		
	Autonomous in flight reconfiguration capacity	Nice to have	10		
Payload	UAV can carry 10kg of payload	Important	1	UAV can fly with and without this payload. Its mass must be proven to the jury.	
	UAV can carry 10kg of payload	Nice to have	1	UAV can fly with and without this payload. Its mass must be proven to the jury.	
	UAV can air drop a payload on a point target within 10m accuracy	Important	1		
	Within a 1km radius	Nice to have	1		
	Without any human	Nice to have	1		
	Air dropping is autonomous	Nice to have	1		
	Air dropping triggered by pattern recognition	Nice to have	1		
	Trajectory adaptation to drop the payload on the detected pattern	Nice to have	10		
Noise and surveillance	Air dropping from a hand using a kit	Nice to have	1		
	UAV can fly over a single point making less than 70dB over that point	Important	1	1. At least 50 meters of altitude	
	UAV can fly over a quiet area making less than 70dB over that point	Nice to have	1	1. Trajectory: UAV is operational over a given area.	
	UAV can capture ground images	Nice to have	1	Possible to read local 10 on a 10 piece of paper from above 10 meters of altitude	
Connectivity	In real time	Nice to have	1		
	Preparing and sending a flight plan with a smartphone	Nice to have	1		
Air Collaboration	Dynamic interaction with a smartphone	Nice to have	1		
	Getting and/or manipulating map on ground	Nice to have	10		
	Having several autonomous flying platforms in support of the main drone	Nice to have	10		
	The secondary drone can follow the master drone	Nice to have	10		
	The secondary drone can perform its own mission (in collaboration with the master drone)	Nice to have	10		
	The secondary drone is dropped from the master drone	Nice to have	10		
Customization	Automatic mission scheduling taking advantages of each drone strength	Nice to have	10		
	Real-time assembly	Important	1	1. UAV is not a RTN drone.	
	Custom flight control laws	Nice to have	1	1. Depends on the complexity determined by the jury.	
	Custom intelligence and/or commands	Nice to have	1	1. Depends on the complexity determined by the jury.	
	Custom electronic components	Nice to have	1	1. Depends on the complexity determined by the jury.	
	Any relevant speed factor	Nice to have	1	1. Depends on the complexity determined by the jury.	
Project related	Total cost of the project below 1000€	Mandatory	0		
	Dassault's delivery ratings	Important	10	10: 10pts = OK from DA, 9pts = 100% from DA, lower delay.	
	Team's equipment	Important	10	10: 10pts = OK from team, 9pts = 100% from team, lower delay.	
	All of all components used, costs precision	Mandatory	0		
	Summary of all challenges	Mandatory	0		
	Continuous integration	Important	1		
Spots	Must be 100%	Important	10		
	Spots as a whole	Important	10	Handling the rules, controlling the jury, negative attitude towards the other participants, punctuality, etc.	
	Innovation	Important	10	10: Innovation regarding the whole project.	
	Anticipation vs preparation	Important	10	Announcing 10 features but doing 1 of them correctly = 10/10 / 1. Doing perfectly everything that has been promised = 10/10	

Algorithme de détection pour drone

MARVIN Leo

5 décembre 2025

1 Introduction

Ce document presente le fonctionnement d'un programme permettant de detecter des obstacles a l'aide d'une camera et de renvoyer une video mettant en valeur les objets detectes.

2 Principe général

Le systeme repose sur l'exploitation de la bibliotheque OpenCV compatible avec Python. Cette bibliotheque permet de pouvoir reconnaitre les differents objets parmi une liste de possibilites de 80 classes.

3 Code informatique

```
import cv2
from ultralytics import YOLO
import math
import random
import numpy as np

# ----- CONFIGURATION -----
# Charge le modele.
model = YOLO('yolov8n.pt') # Modele pre-entraîne sur COCO (80 classes)

# Source de la video
# Mettre 0 pour la webcam, le chemin d'un fichier "video.mp4",
# ou l'URL RTSP du drone (ex: "rtsp://192.168.1.1:554/stream")
#video_source = 0
video_source = "personne_test2.jpg" # Image test

# Recuperer toutes les classes disponibles dans le modele
CLASS_NAMES = model.names # Dictionnaire {id: nom_classe}
print(f"Modele charge avec {len(CLASS_NAMES)} classes disponibles")

# Generer une couleur unique pour chaque classe
def generate_colors(num_classes):
    colors = {}
    for class_id in range(num_classes):
        # Generer des couleurs vives et variees
```

1

```
        hue = (class_id * 137) % 360 # Utiliser l'angle d'or pour une
            bonne repartition
        saturation = 0.7 + random.random() * 0.3 # Entre 0.7 et 1.0
        value = 0.8 + random.random() * 0.2 # Entre 0.8 et 1.0

        # Convertir HSV vers BGR pour OpenCV
        hsv_color = cv2.cvtColor(
            np.uint8([[hue / 2, saturation * 255, value * 255]]), #
            cv2.COLOR_HSV2BGR
        )
        colors[class_id] = tuple(map(int, hsv_color[0][0]))
    return colors

# Generer les couleurs pour toutes les classes
COLORS = generate_colors(len(CLASS_NAMES))

# ----- MAIN LOOP -----
import numpy as np

cap = cv2.VideoCapture(video_source)

if not cap.isOpened():
    print("Erreur : Impossible d'ouvrir le flux video.")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Faire la detection sur l'image
    results = model(frame, stream=True)

    # Traiter les resultats
    for r in results:
        boxes = r.boxes
        if boxes is not None:
            for box in boxes:
                # 1. Recuperer les coordonnees de la boite
                x1, y1, x2, y2 = box.xyxy[0]
                x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

                # 2. Recuperer la classe detectee
                cls_id = int(box.cls[0])

                # 3. Recuperer le nom de la classe depuis le modele
                if cls_id in CLASS_NAMES:
                    class_name = CLASS_NAMES[cls_id]
                else:
                    class_name = f"Classe {cls_id}"

                # 4. Recuperer la couleur pour cette classe
                if cls_id in COLORS:
                    color = COLORS[cls_id]
                else:
                    color = (0, 0, 255) # Rouge par default
```

2

```

# 5. Recuperer le score de confiance
conf = math.ceil((box.conf[0] * 100)) / 100

# 6. Affichage
# Dessiner le rectangle
cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

# Creer le texte a afficher
text_display = f"{class_name} {conf:.2f}"

# Calculer la taille du texte pour le fond
font = cv2.FONT_HERSHEY_SIMPLEX
font_scale = 0.6
thickness = 2
text_size = cv2.getTextSize(text_display, font,
                             font_scale, thickness)[0]

# Dessiner le fond du texte
text_bg_y1 = max(y1 - text_size[1] - 10, 0)
cv2.rectangle(frame,
              (x1, text_bg_y1),
              (x1 + text_size[0] + 10, y1),
              color,
              -1)

# Afficher le texte
cv2.putText(frame,
            text_display,
            (x1 + 5, y1 - 5),
            font,
            font_scale,
            (255, 255, 255), # Texte en blanc
            thickness)

# Afficher le nombre de detections
detection_count = len(boxes) if boxes is not None else 0
cv2.putText(frame,
            f"Detections: {detection_count}",
            (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.8,
            (0, 255, 255),
            2)

# Sauvegarder l'image traitee
cv2.imwrite("Test_personne2_detected.jpg", frame)

# Afficher la video/image
cv2.imshow('YOLO - Detection de toutes les classes', frame)

# Redimensionner la fenetre si l'image est trop grande
height, width = frame.shape[:2]
if width > 1280 or height > 720:
    frame_resized = cv2.resize(frame, (1280, 720))
    cv2.imshow('YOLO - Detection de toutes les classes',
               frame_resized)

```

```

# Quitter avec la touche 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Liberer les ressources
cap.release()
cv2.destroyAllWindows()

# Afficher un resume des classes detectees
if boxes is not None:
    print("\n=== R'\ESUM\ 'E DES D'\ETECTIONS ===")
    unique_classes = {}
    for box in boxes:
        cls_id = int(box.cls[0])
        if cls_id in CLASS_NAMES:
            class_name = CLASS_NAMES[cls_id]
            if class_name not in unique_classes:
                unique_classes[class_name] = 0
            unique_classes[class_name] += 1

    for class_name, count in unique_classes.items():
        print(f"{class_name}: {count} detection(s)")

# Generation d'une seule image
cv2.imwrite("Test_personne3.jpg", frame)

# Generation de la video
# frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
# frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
# fps = cap.get(cv2.CAP_PROP_FPS) # Images par seconde
#
# fourcc = cv2.VideoWriter_fourcc(*'mp4v')
#
# out = cv2.VideoWriter('video_traitee.mp4', fourcc, fps, (frame_width,
# frame_height))

# cap.release()
# cv2.destroyAllWindows()

```

Listing 1 – Capture de la video, detection des points d'interet et renvoi de la video annotee

Annexe 3 : Liste des composants



Universal Smart Charger 1A for Li-Ion/Polymer Battery Pack (3.7V-14.8V 1-4 Cell)

SKU: RB-Ten-28

€36,30

-
1
+

Remove

 **LYNXMOTION**
 **11.1V, 3500mAh, 30C LiPo Battery**
SKU: RB-Ste-02

HQPROP

10" x 4.5" Black Multi-Rotor CW Propeller Pair

SKU: RB-Hqp-16

€6,35

This product is not available in the requested quantity. 3 of the items will be backordered.

3 of the items will be backordered.

- 4 + Remove



LIGPOWER

LIGPOWER Air 40A 2-6S Multi-Rotor UAV Drone ESC

SKU: RB-Tmo-31

€34,24

This product is not available in the requested quantity. 4 of the items will be backordered.

4 of the items will be backordered.

Re-stocking soon

—

4

+

Remove



FLIGHT-RC

iFlight XING2 2207 FPV Motor Unibell 2755KV

SKU: RB-IF-130

— 4 +

Remove

€22,90

This product is not available in the requested quantity. 4 of the items will be backordered.
4 of the items will be backordered.

On Demand

* Delays depend on the order frequency from each supplier and the approximate arrival date is indicated in the due date. Returns are not accepted on On Demand and Clearance items except when they are found defective, in which case the product may be repaired or replaced at RobotShop's discretion.

€91,80

 SEEDSTUDIO Raspberry PI GPS Module SKU: RB-See-570	€40,32	1	Remove	€40,32
--	---------------	----------	---------------	---------------

	RASPBERRY PI Raspberry Pi 4 Compute Module 4 - 16GB eMMC, 2GB LPDDR4 RAM (CM4002016) SKU: RP-See-1104 €65,40 - 1 + Remove	- 1 + Remove €65,40
---	--	---



RADIOLINK ELECTRONIC

RadioLink PIX6 Flight Controller

SKU: RB-RK-56

-

1

+

Remove

€185,45

Returns are not accepted on Special Order and Clearance items except when they are found defective, in which case the product may be repaired or replaced at RobotShop's discretion.

-

1

+

Remove

A screenshot of a shopping cart item. On the left is a small image of a camera module. To its right, the text reads: "ARDUCAM", "ArduCam 8MP IMX219 Wide Angle Camera Module for NVIDIA Jetson Nano", and "SKU: RB-Adu-219". Below this, the price "€21,80" is shown next to a quantity selector (minus, 1, plus) and a "Remove" button. To the right of this, another quantity selector (minus, 1, plus) and a "Remove" button are shown, followed by the price "€21,90".


RADIOLINK ELECTRONIC
RadioLink T8FB(BT) 2.4GHz 8CH Transmitter w/ R8EF 8CH Receiver
 SKU: RB-R8K-40
 €59,99

Détection d’obstacles et PID d’évitement pour drone

MARVIN Leo

5 décembre 2025

Table des matières

1 Introduction

Ce document présente le fonctionnement d’un programme permettant de détecter des obstacles à l’aide d’une caméra et de calculer une commande d’évitement via un régulateur PID. Cette approche est adaptée aux drones ou robots légers, sans recours à l’intelligence artificielle.

2 Principe général

Le système repose sur la différence entre images successives capturées par la caméra. L’idée principale est de détecter les zones en mouvement ou qui s’agrandissent dans l’image, indiquant un obstacle proche, puis de générer une correction pour l’éviter.

3 Code Informatique

```
import cv2
import numpy as np
import time
import argparse
from collections import deque

# -----
# Configuration / Parametres
# -----
CONFIG = {
    # optical flow
    "fb_pyr_scale": 0.5,
    "fb_levels": 3,
    "fb_winsize": 15,
    "fb_iterations": 3,
    "fb_poly_n": 5,
    "fb_poly_sigma": 1.2,
    "fb_flags": 0,

    # zone d interet (ROI) : portion de l'image correspondant "devant"
    # le drone
    # format fractions (y1,y2,x1,x2) ; par ex. (0.2,0.8,0.25,0.75)
```

```
"roi": (0.25, 0.75, 0.25, 0.75),

# seuils de detection
"magnitude_threshold": 3.5, # magnitude moyenne au-dessus de
                             # laquelle on considere un mouvement important
"movement_pixel_count_threshold": 2000, # nombre de pixels
                                         # superieurs pour alerter

# PID parameters (pour commande laterale / yaw)
"pid_kp": 0.8,
"pid_ki": 0.02,
"pid_kd": 0.15,
"pid_integrator_limit": 1.0, # limite pour integrateur (anti-windup
                             # )

# commande (valeurs arbitraires normalisees -1..1)
"max_yaw_command": 0.6, # yaw left/right max
"max_forward_command": 1.0, # forward thrust command (positive
                             # forward)

# temporal smoothing
"smooth_history_len": 5
}

# -----
# PID
# -----
class PID:
    def __init__(self, kp, ki, kd, integrator_limit=None):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.integrator = 0.0
        self.prev_error = None
        self.integrator_limit = integrator_limit

    def reset(self):
        self.integrator = 0.0
        self.prev_error = None

    def update(self, error, dt):
        if dt <= 0:
            derivative = 0.0
        else:
            derivative = 0.0 if self.prev_error is None else (error -
                                                                self.prev_error) / dt

        self.integrator += error * dt
        if self.integrator_limit is not None:
            # clamp integrator
            if self.integrator > self.integrator_limit:
                self.integrator = self.integrator_limit
            if self.integrator < -self.integrator_limit:
                self.integrator = -self.integrator_limit

        out = self.kp * error + self.ki * self.integrator + self.kd *
            derivative
```



```

        self.prev_error = error
        return out

# -----
# Helpers
# -----
def get_roi_rect(frame_shape, roi_frac):
    h, w = frame_shape[:2]
    y1 = int(roi_frac[0] * h)
    y2 = int(roi_frac[1] * h)
    x1 = int(roi_frac[2] * w)
    x2 = int(roi_frac[3] * w)
    return (x1, y1, x2, y2)

def normalize(v, vmin, vmax):
    if vmax == vmin:
        return 0.0
    return float((v - vmin) / (vmax - vmin))

# -----
# Main pipeline
# -----
def main(args):
    # Initialisation source video / camera
    if args.video:
        cap = cv2.VideoCapture(args.video)
    else:
        cap = cv2.VideoCapture(int(args.camera))

    if not cap.isOpened():
        print("Erreur : impossible d'ouvrir la source video/camera.")
        return

    # Lire la premiere frame
    ret, prev_frame = cap.read()
    if not ret:
        print("Erreur : video vide.")
        return

    prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

    # PID for lateral/yaw control
    pid = PID(CONFIG["pid_kp"], CONFIG["pid_ki"], CONFIG["pid_kd"],
              integrator_limit=CONFIG["pid_integrator_limit"])

    # smoothing for commands
    yaw_history = deque(maxlen=CONFIG["smooth_history_len"])
    forward_history = deque(maxlen=CONFIG["smooth_history_len"])

    last_time = time.time()

    # loop
    while True:
        ret, frame = cap.read()
        if not ret:

```

3

```

        break

    ts = time.time()
    dt = ts - last_time if last_time is not None else 0.033
    last_time = ts

    # resize if too large (optional: speed vs accuracy)
    max_dim = 1280
    h, w = frame.shape[:2]
    if max(h, w) > max_dim:
        scale = max_dim / max(h, w)
        frame = cv2.resize(frame, (int(w * scale), int(h * scale)))
        prev_gray = cv2.resize(prev_gray, (frame.shape[1], frame.
                                         shape[0]))

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # optical flow (Farneback)
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None,
                                         pyr_scale=CONFIG["
                                             fb_pyr_scale"],
                                         levels=CONFIG["fb_levels"],
                                         winsize=CONFIG["fb_winsize"
                                                         ],
                                         iterations=CONFIG["
                                             fb_iterations"],
                                         poly_n=CONFIG["fb_poly_n"],
                                         poly_sigma=CONFIG["
                                             fb_poly_sigma"],
                                         flags=CONFIG["fb_flags"])

    mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
    # focus on ROI (forward-looking area)
    roi_rect = get_roi_rect(frame.shape, CONFIG["roi"])
    x1, y1, x2, y2 = roi_rect
    mag_roi = mag[y1:y2, x1:x2]

    # compute simple statistics
    mean_mag = float(np.mean(mag_roi))
    median_mag = float(np.median(mag_roi))
    # count of pixels above certain magnitude
    moving_pixels = int((mag_roi > CONFIG["magnitude_threshold"]).
                        sum())

    # detect approximate centroid of high motion (to decide
    # direction)
    mask = (mag_roi > CONFIG["magnitude_threshold"]).astype(np.uint8)
    * 255
    M = None
    centroid_x_norm = 0.5 # default center
    if mask.sum() > 0:
        M = cv2.moments(mask)
        if M["m00"] != 0:
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])
            # normalize cx to [-1, 1] (left=-1, right=1)
            roi_w = (x2 - x1)
            centroid_x_norm = (cx / float(roi_w) - 0.5) * 2.0

```

4

```

        else:
            centroid_x_norm = 0.0

# Collision decision (simple threshold logic)
collision_alert = False
if moving_pixels > CONFIG["movement_pixel_count_threshold"] and
    mean_mag > CONFIG["magnitude_threshold"]:
    collision_alert = True

# Compute avoidance command with PID:
# Error is centroid_x_norm (positive -> object to the right ->
# we want to yaw left, negative vice-versa)
# We'll invert sign so positive error = need to yaw right.
pid_error = -centroid_x_norm # choose sign convention for
    control (tune as needed)
yaw_cmd_raw = pid.update(pid_error, dt)

# scale and clamp yaw command to [-max, max]
max_yaw = CONFIG["max_yaw_command"]
yaw_cmd = float(max(-max_yaw, min(max_yaw, yaw_cmd_raw)))

# forward command reduction based on mean magnitude (if object
    close, reduce forward speed)
# normalize mean_mag to [0..1] using an expected mag range
# NOTE: tune these expected values for your camera/speed
mag_expected_min = 0.0
mag_expected_max = 10.0 # adjust according to your observations
normalized_mag = normalize(mean_mag, mag_expected_min,
    mag_expected_max)
# if collision alert -> strong braking, otherwise forward
    proportional to (1 - normalized_mag)
if collision_alert:
    forward_cmd = float(-0.3) # negative = slow/stop/backoff;
        tune with care
else:
    forward_cmd = float(CONFIG["max_forward_command"] * (1.0 -
        normalized_mag))
# clamp
forward_cmd = max(-CONFIG["max_forward_command"], min(CONFIG
    ["max_forward_command"], forward_cmd))

# smoothing
yaw_history.append(yaw_cmd)
forward_history.append(forward_cmd)
yaw_cmd_s = float(np.mean(yaw_history))
forward_cmd_s = float(np.mean(forward_history))

# For debug / simulation: print and overlay
info_text = f"mean_mag={mean_mag:.2f} med={median_mag:.2f}
    mov_px={moving_pixels} ALERT={collision_alert}"
cmd_text = f"YAW={yaw_cmd_s:.2f} FORWARD={forward_cmd_s:.2f}"
# draw ROI
vis = frame.copy()
cv2.rectangle(vis, (x1, y1), (x2, y2), (200, 200, 0), 2)
# overlay mask scaled to full image for visualization
mask_full = np.zeros_like(gray)
mask_full[y1:y2, x1:x2] = mask
mask_rgb = cv2.cvtColor(mask_full, cv2.COLOR_GRAY2BGR)

```

5

```

vis = cv2.addWeighted(vis, 0.8, mask_rgb, 0.6, 0)

# draw centroid
if M is not None and M["m00"] != 0:
    cx_img = x1 + int((centroid_x_norm/2.0 + 0.5) * (x2 - x1))
    cy_img = y1 + int((0.5) * (y2 - y1))
    cv2.circle(vis, (cx_img, cy_img), 6, (0, 0, 255), -1)

# print texts
cv2.putText(vis, info_text, (10, 20), cv2.FONT_HERSHEY_SIMPLEX,
    0.55, (255, 255, 255), 1)
cv2.putText(vis, cmd_text, (10, 45), cv2.FONT_HERSHEY_SIMPLEX,
    0.6, (0, 200, 0) if not collision_alert else (0, 0, 255), 2)

if collision_alert:
    cv2.putText(vis, "COLLISION ALERT!", (10, 80), cv2.
        FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 3)

cv2.imshow("optical_flow_collision_pid", vis)

# ---- Here you would send commands to your flight controller /
    autopilot ----
# Example (simulation): print or log
print(f"[{time.strftime('%H:%M:%S')}] {info_text} | CMD: yaw={
    yaw_cmd_s:.2f}, forward={forward_cmd_s:.2f}")

# Example MAVROS integration stub (uncomment and adapt if using
    ROS / MAVROS):
# import rospy
# from geometry_msgs.msg import Twist
# twist = Twist()
# twist.linear.x = forward_cmd_s # forward/back speed
# twist.angular.z = yaw_cmd_s # yaw rate
# vel_pub.publish(twist)

# handle user keyboard
key = cv2.waitKey(1) & 0xFF
if key == 27: # ESC
    break
if key == ord('r'):
    pid.reset()

prev_gray = gray

cap.release()
cv2.destroyAllWindows()

# -----
# CLI
# -----
if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    group = ap.add_mutually_exclusive_group(required=False)
    group.add_argument("--video", help="Path to video file (e.g. videos/
        test.mp4)")
    group.add_argument("--camera", help="Camera index (default 0)",
        default="0")

```

6

```
ap.add_argument("--show", help="Show display (default true)", action  
               ="store_true")  
args = ap.parse_args()  
main(args)
```